Multi-Agent AI Lab: LLM Agents, Retrieval and Resource-Aware Cooperation

Language models and classical methods

Session 3 · Theory

Julia Schaumeier

IN2106 · WS 2026/27 · 2 November 2026



Today

1. Last week, revisited
2. Two kinds of problem
3. Classical methods
4. The language model as a component
5. The language model as an agent
6. Retrieval
7. Choosing and measuring

Last week, revisited

	Clear	Muddy	
Why several agents	when one agent is enough and when several are needed		
Games	how agents decide: dominant strategies, Nash equilibria	can anything make sure an equilibrium is reached? the stranger in Munich	Moodle
Coordination	the protocols, how they differ, how agents coordinate; roles and tasks; the kitchen		
The Archive Games		what the game is, exactly; how the maze and the archive work	handout, Session 4
Building it		how complex the specialised agents get; design details	today, Session 4

One of you wrote: everything was clear.

By the end of today, you will be able to

1. say for one decision in an agent, such as labelling a review or choosing a move, whether a classical method or a language model fits, and what each needs
2. explain what happens in a model call, and why the reply must be checked
3. describe how tools, a loop and memory turn a language model into an agent
4. explain retrieval as ranking, and why an answer must be checked against its evidence
5. plan a fair comparison of two methods, with cost as part of the result

Two kinds of problem

Not every problem is a language problem

Language problems	Structured problems
find the relevant document; answer a question from text; extract a name or a figure; classify a text; check whether a text supports a claim	find a route; pack a lorry; solve a puzzle; choose a move against an opponent; allocate tasks

Classical methods exist for both. Language models are now applied to both.

Would you ask a language model?

Hands up: yes — no — depends

- to build the timetable of TUM's computer science master's course
- to sort the CIT Pulse survey into classes of feedback

Remember your answer: we come back to both later today.

Classical methods

Classical methods for language

Method	Example
Words and their weights: which words a text contains, and how rare each is in the collection	in 1,000 phone complaints, “cracked” sets a few apart; “phone” is in all of them
Patterns: regular expressions, templates, rules	`\b[0-9]{5}\b` finds five digits standing alone, such as the postcode 80333 in an address
Named entities: people, places, dates, numbers	Marie Curie (person) moved to Paris (place) in 1891 (date)
Text classifiers: trained on word counts, to sort texts into classes	a spam filter, trained on emails people marked as spam or not
String similarity: how alike two names or titles are	“Meier” and “Meyer” are one letter apart: perhaps the same person

Typically cheap, transparent and repeatable — and brittle when the wording changes.

Jurafsky & Martin (2026, Chapter 4 and Sections 2.6, 2.9, 11.1 and 18.3)

Classical methods for structured problems

Method	Example
Search over states	a satnav: the state is where you are, an action the next road, the goal the destination
Constraint satisfaction over assignments	a seating plan: each guest gets a table, and two guests who quarrel never share one
Game-tree search: a move against an opponent, looking several turns ahead	tic-tac-toe: try each move, each reply and each reply to that; pick the move whose worst outcome is best
Optimisation: linear and integer programming, local search	a bakery: how many loaves and cakes to bake with the oven time and flour it has, for the most profit

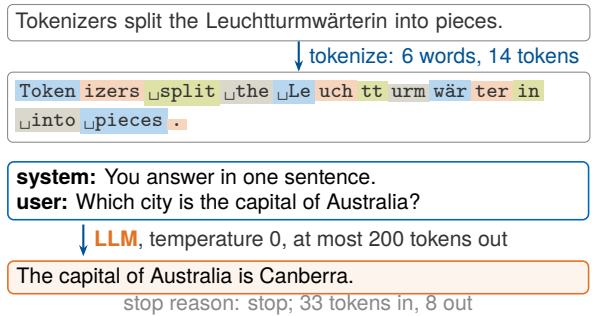
Typically exact or provably good, and fast — once someone has stated the problem formally.

Russell & Norvig (2021, Chapters 3–6)

The language model as a component

One call

- A language model (LLM) writes text by predicting the next **token**, over and over
- A token is a word or a piece of one
- A call sends the **prompt**: a list of messages, here a system message with instructions and a user message with the request
- The **context window** caps input and output tokens together
- Nothing carries over between calls: a conversation is sent again in full



What goes into a prompt

- **Role**, who the model acts as
- **Task**, stated once and plainly
- **Context**, the material to work from
- **Examples**, a few worked cases (few-shot)
- **Output**, the shape the answer should take

system: You sort product reviews. Be brief.
user: Is the review positive or negative?
"Broke after a week." -> negative
"Does exactly what it promises." -> positive
Review: "Arrived late, but works perfectly."
Answer with one word: positive or negative.

↓ **LLM**, temperature 0

positive

By convention, the **system message** holds what stays the same on every call: the role, rules that always apply, the tone, often the output format.

The same prompt, different answers

- For each next token the model gives a probability for every possible token; one is drawn
- **Temperature** sets how much the draw varies: at 0 the likeliest token every time (left), higher more variety (right)
- Even at 0, two runs now and then differ: the service computes many requests at once, and rounding can tip a close choice between two tokens. So one run proves little

system: You are a novelist.
user: Write the opening sentence of a story about a lighthouse, in at most 8 words. Reply with the sentence alone.

LLM, temperature 0, three times

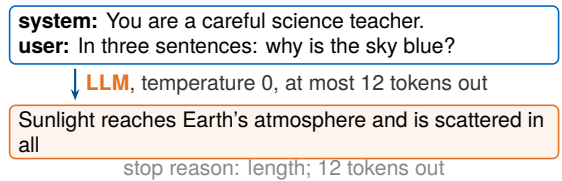
1. The light flickered, then finally went dark.
2. The light flickered, then finally went dark.
3. The light flickered, then finally went dark.

LLM, temperature 1, three times

1. The light breathed, pulsing against the void.
2. The light flickered, then finally went dark.
3. The light died on a Tuesday.

Settings that make results comparable

- **Model**: name it exactly, with its version, and record it
- **Temperature**: set it explicitly, and every other setting you can; what you leave out, the server decides
- **Seed**: makes sampling repeatable, where the service supports it
- **Maximum output length**: an answer cut off looks like a wrong answer



Log every call: the prompt, the reply, the **stop reason** ("stop" when finished, "length" when cut off) and the **token counts**, as under the reply above

Chain of thought

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?
A: The answer is 11.
Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

↓ LLM

A: The answer is 27. **wrong**

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?
A: **Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. 5 + 6 = 11.** The answer is 11.
Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

↓ LLM

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had 23 - 20 = 3. They bought 6 more apples, so they have 3 + 6 = 9. The answer is 9. **right**

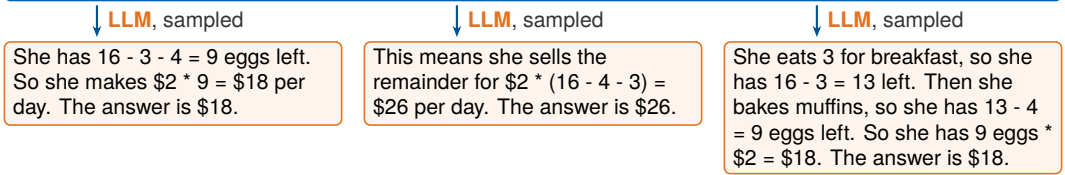
If the example in the prompt spells out its steps, the model spells out its steps too before it answers. That helps on problems with several steps, for the price of a longer prompt and a longer answer.

Wei et al. (2022, Figure 1)

Self-consistency

Q: If there are 3 cars in the parking lot and 2 more cars arrive, how many cars are in the parking lot?
A: There are 3 cars in the parking lot already. 2 more arrive. Now there are 3 + 2 = 5 cars. The answer is 5.

Q: Janet's ducks lay 16 eggs per day. She eats three for breakfast every morning and bakes muffins for her friends every day with four. She sells the remainder for $2 per egg. How much does she make every day?
A:



Sample several chains of thought and take the most common answer, here $18. The chains come from one model, so their mistakes are not independent: the vote evens out a slip in one chain, not an error the model makes every time.

Wang et al. (2023, Figure 1)

The model writes text; your program acts

- Ask for a fixed format, and check the reply in code before you use it: is the order on record, is the request allowed?
- If it fails the check, send the reply back with what is wrong and ask again, a few times at most
- Then fall back on a safe choice made by your program
- Writing a rule into the prompt does not make it hold: only your program can guarantee it

```
ALLOWED = ["refund", "replacement"]

def read_complaint(email):
    prompt = ("Reply as: ORDER <number> REQUEST "
              "<refund or replacement>\n" + email)
    for _ in range(3):        # three tries at most
        reply = llm(prompt)
        order, request = parse(reply)
        if order in orders and request in ALLOWED:
            return order, request
        prompt += "\nYour reply: " + reply
        prompt += "\nUnknown order or request."
    return None, "pass to a person"   # default
```

Classical or language model?

5 minutes — with a neighbour

Take one problem from each column. For each, sketch two programs: one with a classical method, one that asks a language model.

Structured

- **Timetable**: the courses of your computer science master's. Give every lecture a time slot and a hall, with no lecturer or hall booked twice and no clash between courses that students often take together
- **Chess**: a game in progress, your move. Choose one

Language

- **Contract**: a 40-page supplier contract. Find the clause that limits how much either side must pay if something goes wrong
- **Reviews**: 5,000 reviews of a kettle in an online shop. Label each one positive or negative

For each program: what does it need from you, and what could go wrong?

Classical or LLM? Possible answers

Problem	Classical	Language model
Timetable	a constraint solver: time slots and halls as variables, the rules as constraints; needs every rule stated formally and ignores any rule left out	the rules in words; may book a lecturer twice, so a check in code goes over the result
Chess	game-tree search with the rules built in; needs a way to score a position, and plays badly if the score is poor	the position as text; may suggest an illegal move, so the program checks it against the rules
Contract	patterns for phrases such as “shall not be liable”, or a classifier trained on labelled clauses; misses a paraphrase	reads the wording; costs tokens for every page and may point to the wrong clause
Reviews	a keyword list or a classifier trained on labelled reviews; cheap, but blind to irony	a prompt with a few examples; reads nuance, costs tokens for each of 5,000 reviews and may answer differently each time

Other answers are fine if argued.

Two kinds of method, side by side

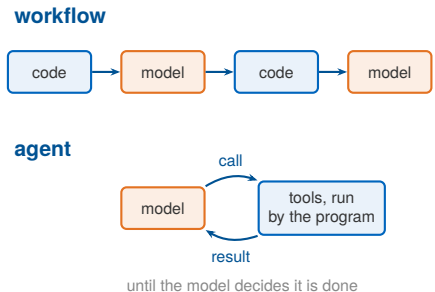
	classical	language model
needs	a formal statement of the problem	a description in words
cost per decision	usually small	tokens, time, money
repeatable	yes	approximately
when it is wrong	often visibly	fluently and confidently
new kinds of input	breaks	often copes

- Neither kind of method is better in general
- Choose for each decision your agent makes, not once for the whole agent: one agent can use both
- Which one fits a decision is settled by measuring — the last part of today

The language model as an agent

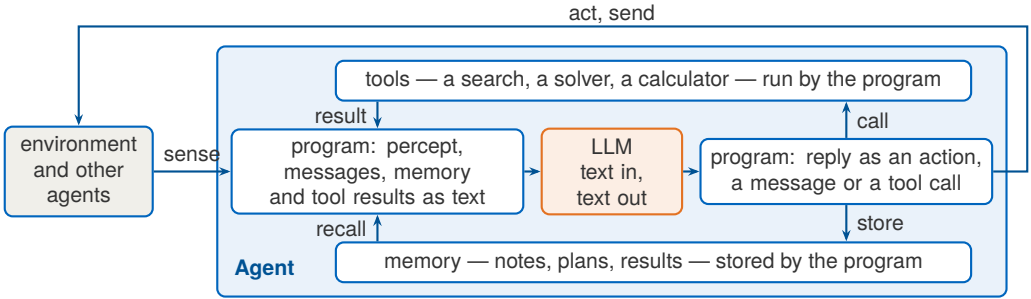
Workflows and agents

- **Workflow**: the path is fixed in code; the model fills in steps
- **Agent**: the model chooses the path as it goes
- Start with the simplest thing that works; add autonomy only where you can show it pays



Anthropic (2024)

From one call to an agent



- As in Session 2: the model only reads and writes text; the program does everything else
- **Tools**: the model asks for a call; the program runs it and adds the result to the prompt
- **Memory**: the prompt holds what this step needs; the program stores the rest
- **Planning**: the model writes a plan as text and revises it as results come in

Weng (2023)

Tool use

system: Reply with one line only:
calculator[EXPRESSION] to have something computed, or finish[ANSWER] when you know the answer.
user: A cinema has 37 rows of 48 seats. 1,129 tickets are sold. How many seats are empty?

↓ **LLM**, temperature 0

calculator[37 * 48 - 1129]

your program runs the tool:
calculator[37 * 48 - 1129] = 647

↓ and calls the model again

system: (as above)
user: A cinema has 37 rows of 48 seats. 1,129 tickets are sold. How many seats are empty?
Your reply: calculator[37 * 48 - 1129]
Observation: 647

↓ **LLM**, temperature 0

finish[647]

The model only proposes a call; your program runs it and puts the result into the next prompt. A tool can be a classical method: a search, a solver, a calculator.

Schick et al. (2023)

ReAct: reasoning and acting, interleaved

Question: In which city was the inventor of the telephone born?
Thought: I need to know who invented the telephone.
Action: search["telephone inventor"]
Observation: Alexander Graham Bell is credited with …
Thought: Now I need Bell's birthplace.
Action: search["Alexander Graham Bell"]
Observation: Born 3 March 1847 in Edinburgh, Scotland …
Thought: Bell was born in Edinburgh.
Action: finish["Edinburgh"]

The model plans one step at a time: each observation decides the next action. **Orange**: written by the model. **Blue**: put into the prompt by your program, which runs each action.

Yao et al. (2023)

Exactly or roughly?

1 minute — answers from the room

A customer-service agent handles a long complaint.

- What must it remember exactly?
- What can it remember roughly?

Memory

- What goes back into the prompt at each step?
  - the full transcript: every detail, but it grows with each step
  - a summary or a selection: fewer tokens, but it can drop what matters
  - state kept in code, such as an order number: exact, but only what you planned for
- **Long-term memory**: notes stored outside the prompt, and only the relevant ones fetched back — retrieval, the next part
- **Shared memory**: several agents read and write one store, as on Session 2's blackboard
- **Reflexion**: after a failed attempt, the model writes down what went wrong; the note goes into the next attempt

Weng (2023); Shinn et al. (2023)

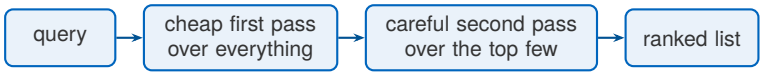
Retrieval

Retrieval is ranking

Return the texts of a collection that best match a query, the most relevant first.

Sparse	Dense
match the words of the query, each weighted by how rare it is in the collection	a neural model turns query and texts into vectors (**embeddings**); similar meanings lie close together
finds the exact word; misses a synonym	finds the synonym; can miss an exact name or number

Often both:



Karpukhin et al. (2020); Jurafsky & Martin (2026, Sections 11.1 and 11.3)

Was the ranking any good?

Query: *how to reset the router password*

rank	page	relevant?
1	Resetting your password	yes
2	Router models compared	no
3	Factory reset	yes
4	Warranty terms	no
5	Wi-Fi channels	no

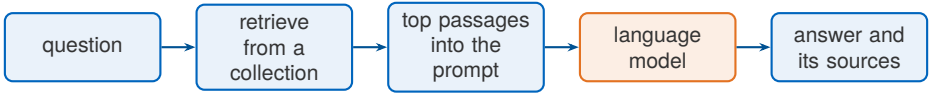
A third relevant page exists in the collection, but it did not make the top 5.

- **Precision**: the share of the returned pages that are relevant
- **Recall**: the share of all relevant pages that were returned
- **At** k: measured over the top k only. Precision at 5: 2 of the top 5 are relevant. Recall at 5: the top 5 hold 2 of the 3
- **The first relevant page**: rank 1 here. It matters most when only the top few pages reach the model

Choose the measure before you build, not after.

Manning et al. (2008, Sections 8.3 and 8.4); Jurafsky & Martin (2026, Section 11.2)

Retrieval-augmented generation



Closed book

the model answers from what it learned in training

no source to point to; knows nothing newer than its training data, and nothing private

Open book

the model answers from passages your program retrieved

can cite them and use current or private text; only as good as what was retrieved, and the model may ignore it

Choices to make: documents or passages; how many go into the prompt; a second ranking pass; retrieving again when the answer is not there yet.

Jurafsky & Martin (2026, Sections 11.3–11.5)

The support assistant

2 minutes — answers from the room

A support assistant answers questions from 10,000 pages of product manuals.
A customer writes: *“My kettle beeps three times and switches off.”*
The manual’s page says: *“Error E3: the water tank is empty.”*

- Would sparse retrieval find that page? Would dense retrieval?
- Whole manuals or passages, and how many go into the prompt?

Answers that need evidence

Finding the right pages is half the job: the answer must also rest on them.

- **Multi-hop**: the answer needs facts from more than one document. The ReAct example was one: first the inventor, then his birthplace
- A correct answer names its **supporting evidence**, the sentences it rests on. Benchmarks score both: a right answer from the wrong sentences loses points
- A language model can **hallucinate**: state something false, fluently, sources included. In *Mata v. Avianca* (2023), lawyers filed court papers citing decisions that ChatGPT had made up

So an answer has to be checked: does the evidence really say what it claims?

Yang et al. (2018); Thorne et al. (2018); *Mata v. Avianca, Inc.*, 678 F. Supp. 3d 443 (S.D.N.Y. 2023)

Checking a claim, two ways

Claim: *Bell was born in Glasgow.* Evidence: *Born 3 March 1847 in Edinburgh.*

Classical	Language model
extract the named entities in the claim — people, places, dates — and compare them with those in the evidence	give it the claim and the evidence, and ask for one label: **supports**, **refutes** or **not enough information**
here: Glasgow against Edinburgh: refuted	*here:* refutes
needs rules for what to extract and what counts as a match	needs the three labels in the prompt, and a check on the reply

The model's task is **natural language inference**, and there are benchmarks for it: FEVER, built from Wikipedia sentences, uses these three labels.

Thorne et al. (2018)

Choosing and measuring

Cost is part of performance

Every choice today — a classical method or a model, one call or an agent — is settled by measuring. Start with what to count.

- Count tokens, calls, time, money and how much text the agent read
- In an agent, cost grows with every step: each call sends the growing prompt again
- An evaluation that reports accuracy without cost is half a result: plot one against the other
- Simple, cheap baselines can match elaborate agents at a fraction of the cost

Kapoor et al. (2024)

Comparing methods fairly

Settings, seeds and output limits alone can move accuracy by several percentage points, and many reported gains depend on such differences. A fair comparison rules them out:

- The same inputs for every method, set aside before you build and never used for tuning
- Baselines: a simple classical method, and the model with a plain prompt
- Change one thing at a time; keep every other setting the same, and record it
- Repeat with several seeds; report the mean and the standard deviation
- Leave room for long answers: a run cut off by a limit is not a finished run
- Log every model call from the first one

Hochlehnert et al. (2025)

A fair comparison

4 minutes — with a neighbour

Back to the 5,000 kettle reviews. You want to know whether a language model labels them positive or negative better than a keyword list does. A person has labelled 600 of them.

- Which labelled reviews do you test on, and which may you use while building?
- Besides the keyword list, what is the simplest version of the model to compare against?
- What do you measure besides accuracy?
- The model's answers vary from run to run: how many runs, and what do you report?

A fair comparison: possible answers

Test and build	set some aside before building, say 200, and test both methods on exactly those; build and tune on the other 400
Simplest model	the model with the plainest prompt: no examples, no chain of thought
Besides accuracy	tokens and time per review; the reviews each method gets wrong, read one by one
Runs and report	several runs of the model at fixed settings, each with its own seed; the mean and the standard deviation. The keyword list needs one run

What to take away

- Language and structured problems each have classical and language-model methods, and each needs something different from you; choose per **decision**
- A model call is **tokens** in, tokens out, with no memory; fix the settings and check every reply in code
- **Tools**, a loop and **memory** make a model an agent; a fixed workflow is often enough
- **Retrieval** is ranking; a model can hallucinate, so an answer is checked against its evidence
- A fair comparison tests on inputs set aside, changes one thing, repeats and reports **cost** with accuracy

One-minute paper

1 minute — on your own

- Orange note: the clearest point today
- Pink note: the muddiest point today
- Leave them on the wall as you go

Further reading

On top of the lists in Sessions 1 and 2, for today and your literature survey:

Weng, L. (2023, June 23). LLM powered autonomous agents. *Lil'Log*. — free online

Anthropic (2024, December 19). Building effective agents. *Anthropic Engineering*. — free online

Yao, S., et al. (2023). ReAct: Synergizing reasoning and acting in language models. arXiv:2210.03629v3.

Lewis, P., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33* (pp. 9459–9474). — free online

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press. — free online

Jurafsky, D., & Martin, J. H. (2026). *Speech and Language Processing* (3rd ed. draft). — free online

Kapoor, S., et al. (2024). AI agents that matter. arXiv:2407.01502v1.

Until we meet again

- **This week**: in M1, your approach compared with at least one alternative, and how you will know it works
- **Mon 9 Nov**, Session 4: the Archive Games — the rules, the API, your code
- **Thu 12 Nov, 2 pm**: M1 due

References

References

Anthropic (2024, December 19). Building effective agents. *Anthropic Engineering*. https://www.anthropic.com/engineering/building-effective-agents

Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2024). Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *The Twelfth International Conference on Learning Representations (ICLR 2024)* (pp. 9112–9141). https://proceedings.iclr.cc/paper_files/paper/2024/file/25f7be9694d7b32d5cc670927b8091e1-Paper-Conference.pdf

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., & Wang, H. (2024). Retrieval-augmented generation for large language models: A survey. arXiv:2312.10997v5.

Hochlehnert, A., Bhatnagar, H., Udandarao, V., Albanie, S., Prabhu, A., & Bethge, M. (2025). A sober look at progress in language model reasoning: Pitfalls and paths to reproducibility. arXiv:2504.07086v2.

Jurafsky, D., & Martin, J. H. (2026). *Speech and Language Processing* (3rd ed. draft, release of August 19, 2026). https://web.stanford.edu/~jurafsky/slp3/

Kapoor, S., Stroebl, B., Siegel, Z. S., Nadgir, N., & Narayanan, A. (2024). AI agents that matter. arXiv:2407.01502v1.

Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W. (2020). Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 6769–6781). Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.emnlp-main.550

References

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33* (pp. 9459–9474). Curran Associates. https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.

Mata v. Avianca, Inc., 678 F. Supp. 3d 443 (S.D.N.Y. 2023).

Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed., global ed.). Pearson.

Shoham, Y., & Leyton-Brown, K. (2010). *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations* (Rev. 1.1). Cambridge University Press.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems 36* (pp. 68539–68551). Curran Associates. https://doi.org/10.52202/075280-2997

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36* (pp. 8634–8652). Curran Associates. https://doi.org/10.52202/075280-0377

References

Thorne, J., Vlachos, A., Christodoulopoulos, C., & Mittal, A. (2018). FEVER: A large-scale dataset for fact extraction and VERification. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)* (pp. 809–819). Association for Computational Linguistics. https://doi.org/10.18653/v1/N18-1074

Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., & Zhou, D. (2023). Self-consistency improves chain of thought reasoning in language models. arXiv:2203.11171v4.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems 35* (pp. 24824–24837). Curran Associates. https://doi.org/10.52202/068431-1800

Weng, L. (2023, June 23). LLM powered autonomous agents. *Lil'Log*. https://lilianweng.github.io/posts/2023-06-23-agent/

Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W., Salakhutdinov, R., & Manning, C. D. (2018). HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing* (pp. 2369–2380). Association for Computational Linguistics. https://doi.org/10.18653/v1/D18-1259

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. arXiv:2210.03629v3.